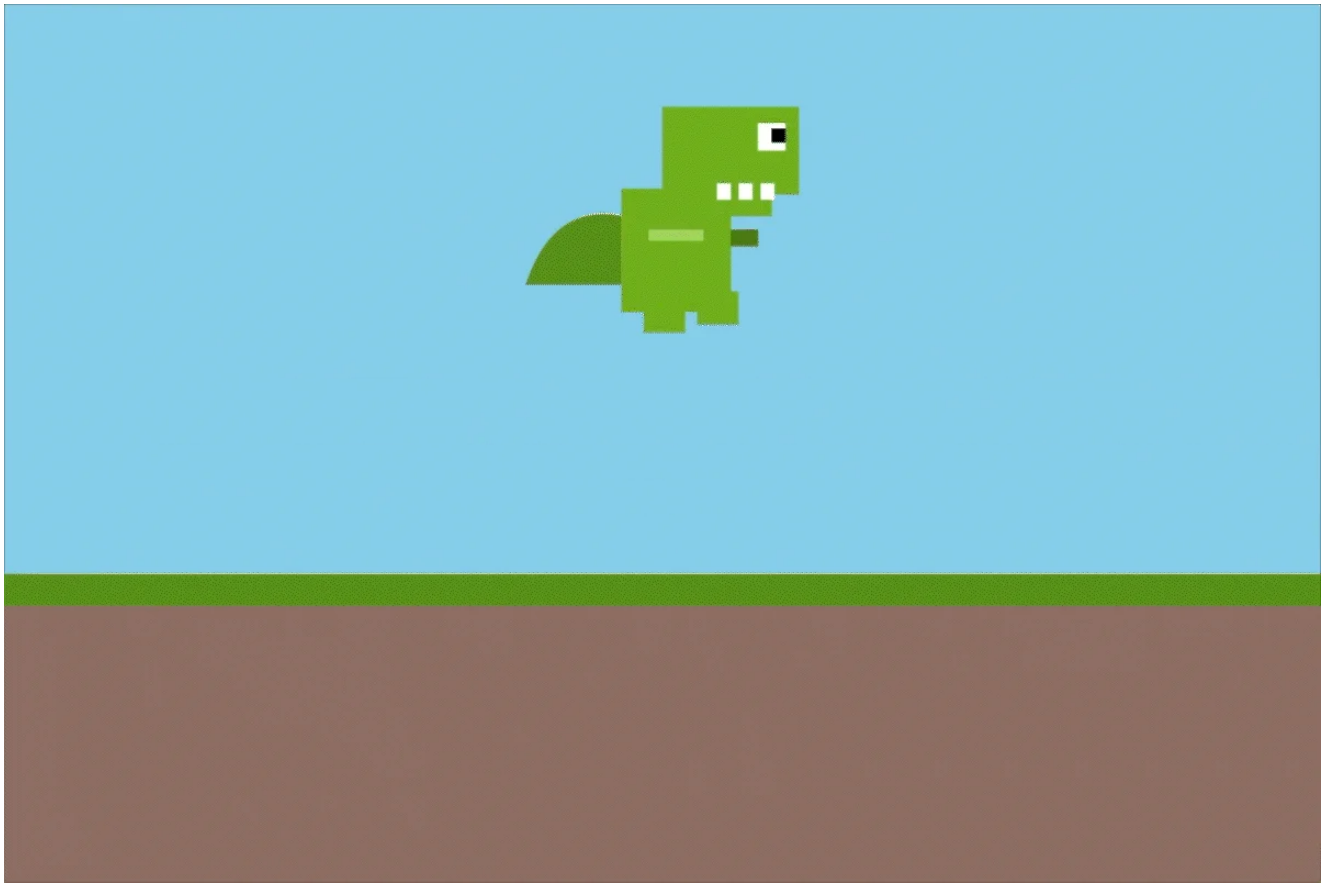


© De Standaard

DE HELPDESK

Vibecoding: zo maak je in enkele minuten je eigen games, websites en apps met AI

Waarom zou je een app gebruiken die geld kost of een abonnement vereist, en toch niet helemaal doet wat je nodig hebt? Met vibecoding maak je een app of website op maat. Of liever: AI maakt die voor je.

Dominique Deckmyn

23 januari 2026 om 23:59

Luisteren Delen

Het is nog maar januari, maar ‘vibecoding’, ofwel coderen met AI, maakt al een goede kans om het woord van 2026 te worden. AI gebruiken om je eigen apps en websites te bouwen is eigenlijk al twee jaar in opmars. Maar de jongste weken lijkt het fenomeen in een stroomversnelling te zijn gekomen. Onder meer omdat de nieuwste versies van

Claude, Gemini en ChatGPT zo betrouwbaar kunnen programmeren, dat je er zonder noemenswaardige voorkennis aan zou kunnen beginnen.

Maar is dat zo? Wel: ja en nee. Ja: een simpel programma, zoals een spelletje dat je in je browser kunt spelen, dat kun je nu werkelijk in een-twee-drie maken met alleen maar een prompt. Denk aan “maak een spel waarbij je cocktails moet mixen”. Al helpt het wel om eerst een paar handigheidjes te leren, wat we hieronder zullen doen.

Wil je iets ambitieuzers ontwikkelen, zoals de gezinskalender-app die ik vorige week bouwde, dan moet je wel een beetje weten waar je mee bezig bent. Je hoeft niet te kunnen programmeren, je hoeft zelfs geen programmacode te kunnen lezen, wel moet je nadenken over wat je precies wilt bouwen en welke componenten daarvoor nodig zijn. Maar hier is het goede nieuws: je kunt klein beginnen en heel snel je grenzen verleggen.

Een eenvoudig spel

Hoe klein? Wel, laten we beginnen met zo’n spelletje. Open een chatbot naar keuze: Gemini, ChatGPT, Copilot, Claude. De gratis versie volstaat. En vraag om een simpel spel. Ik probeerde bijvoorbeeld: “Schrijf een spel waarbij een dinosaurus een bal moet koppen.”

Dit spel in Javascript is gegenereerd door Gemini met één prompt: “Maak een spel waarbij een t-rex een bal moet koppen.”

Zowel Gemini als ChatGPT doet dat meestal in een paar seconden. Beide produceren dan een hoop code in een combinatie van HTML (de opmaaktaal van het web) en de programmeertaal Javascript. Wat doe je met die code? Die bewaar je in een document met de extensie ‘.html’, bijvoorbeeld ‘Mijnspelletje.html’ op je computer.

Op een Windows-pc:

1) Klik op de code die de chatbot genereerde (meestal op het ‘copy’-knopje bovenaan of onderaan)

2) Plak de code in een programma als Windows Kladblok of Macintosh TextEdit

3) Bewaar het bestand met een naam als ‘dinospel.html’

4) Klik op dinospel.html

5) Het spel opent in je standaardbrowser. En spelen maar!

Als je Microsoft Kladblok gebruikt (dat op elke Windows-pc geïnstalleerd zou moeten zijn), moet je wel op één ding letten: het programma heeft de neiging om elk document te bewaren met de extensie “.txt”. Hou dus in de gaten dat hij er niet stiekem dinospel.html.txt van heeft gemaakt.

Ik vibecode het liefst op mijn laptop, maar het kan ook op een smartphone. Wil ik het programma dat ik op mijn pc maakte op mijn smartphone spelen, dan kan ik het bijvoorbeeld naar mijzelf e-mailen en op mijn smartphone de bijlage bewaren.

Lukt dat allemaal elke keer? Nee, maar meestal wel, eigenlijk. Soms begrijpt de chatbot je verkeerd, en moet je wat meer preciseren: “Schrijf het programma in Javascript en zet alles in één bestand.” En je kunt ook aanwijzingen geven over de visuele stijl (‘laat het eruitzien als een 8-bit computerspel’), de bediening (‘met de pijltjestoetsen’) of het spelverloop.

Vaak zit de eerste versie niet helemaal goed: er wordt geen score bijgehouden, bijvoorbeeld, of de voorwerpen in het spel bewegen te snel of juist te traag. Dan doe je een tweede ronde: je vraagt aan ChatGPT of Gemini om de nodige aanpassingen. En hop: een verbeterde versie verschijnt, die je op dezelfde manier op je harde schijf bewaart.

Op die manier krijg je meestal spelletjes die sterk geïnspireerd zijn op wat al bestaat. Maar het is zeker ook mogelijk om meer originele spelletjes te bouwen, als je een meer gedetailleerde beschrijving geeft. Stap voor stap uitbreiden met extra functies of effecten (geluid!) kan ook.

Zo bouwde ik in vijf stappen een ‘cocktailspel’, waarbij je in enkele seconden een cocktail moet mixen op basis van een recept. De eerste versie was wat saai, dus ik voegde er opties als ‘schudden’ en ‘parasolletje’ aan toe.

ChatGPT schreef dit spel in een aantal stappen, na telkens een kleine bijsturing.

Elke keer dat een programma een foutmelding geeft, kleef je die foutmelding in ChatGPT met de vraag om die fout te repareren. Dat lukt vaak meteen, soms moet je een paar keer proberen.

Ambitieuzer: een gezinskalender

Na een tijdje durfde ik iets meer complexe apps aan. Terwijl de AI voor grafische apps zoals spelletjes en websites meestal spontaan naar HTML en Javascript zal grijpen, kom je voor apps met meer tekst of cijfers eerder bij de programmeertaal Python uit. Dat is een prachtige taal om mee aan de slag te gaan, maar je moet wel eerst een werkende Python-omgeving op je computer hebben staan. Installeer gratis de recentste versie van Python via python.org.

Wil je iets met Python uitproberen, dan kun je op dezelfde manier te werk gaan als met Javascript: ChatGPT of Gemini schrijft de code, je plakt die in Kladblok en bewaart hem op je harde schijf – maar nu met de extensie '.py'. Om hem uit te voeren, moet je dan, vanaf de opdrachtprompt, in de juiste map gaan staan en 'python mijnprogramma.py' intikken. Beetje omslachtig, dus.

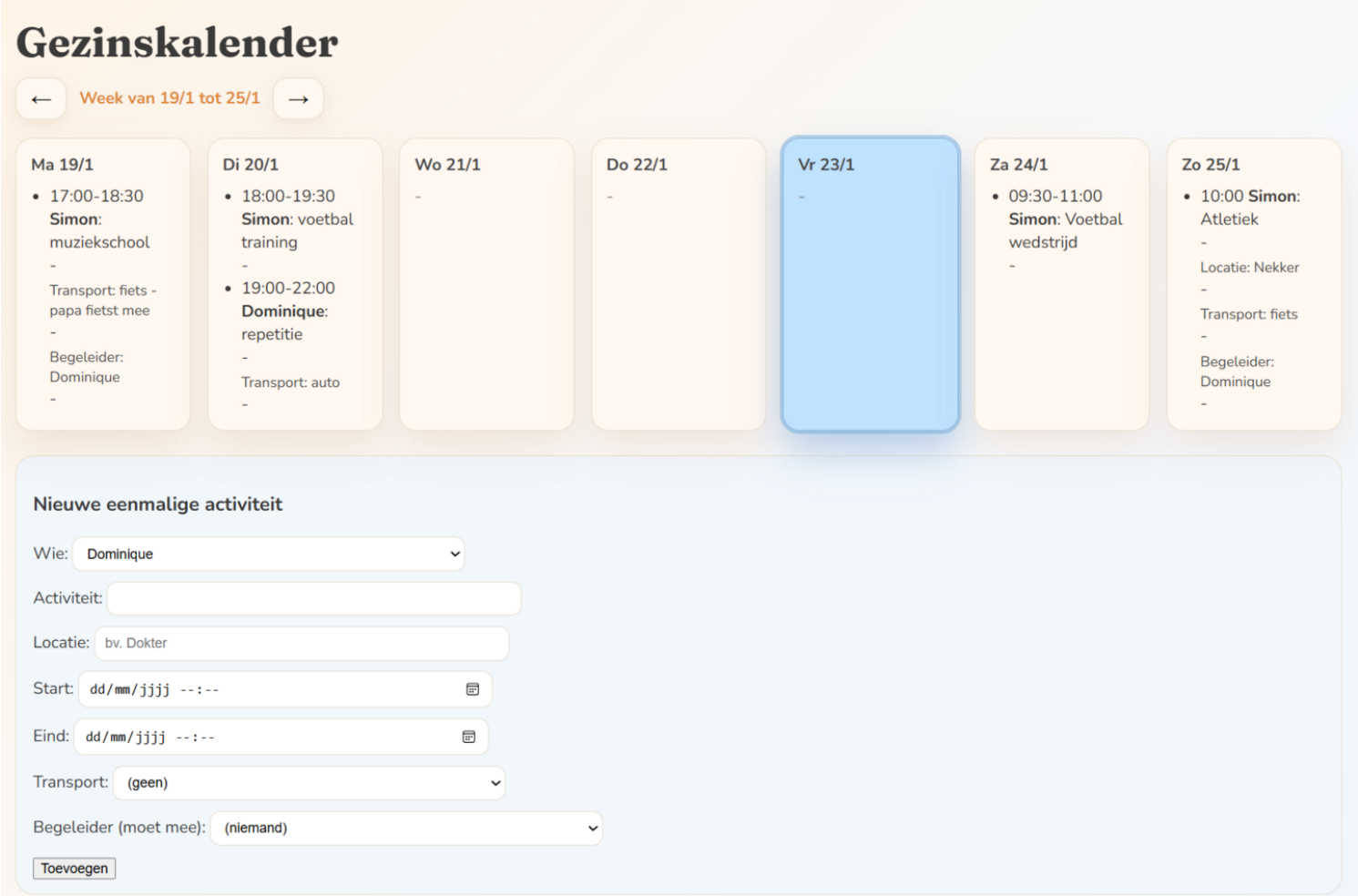
Voor mijn app Gezinskalender gebruikte ik toch maar Javascript. Het moest een app worden voor alle leden van een gezin. Eentje waarin ze elkaars activiteiten zien, maar bijvoorbeeld ook kunnen zien wie van de ouders het zoontje op dinsdagavond naar de voetbaltraining moet brengen. Ieder gezinslid moet een activiteit kunnen toevoegen.

Dat is te ingewikkeld voor een programma van één bestand. De app bestaat dus, opnieuw, uit HTML en Javascript. De tabel met alle activiteiten moet toegankelijk zijn voor iedereen en moet dus ergens centraal worden bewaard. ChatGPT suggereerde een paar opties, ik koos daaruit Supabase, een cloud-database. Om iedereen toegang te geven tot dezelfde versie van het programma, heb ik het 'gehost' op een site die Vercel heet. Op beide sites maakte ik gratis accounts aan. Ik liet me elke stap in detail uitleggen door ChatGPT, het lukte zonder dat ik iets moest leren over Vercel of Supabase.

Ik besliste uiteindelijk, alweer op advies van ChatGPT, om mijn programma niet

DS BO

altijd kunt terugkeren naar een eerdere, werkende versie.



Deze gezinskalender draait op smartphone en pc, de gegevens worden in de "cloud" bewaard. De eerste versie werd geschreven door ChatGPT, maar de afwerking gebeurde met hulp van ChatGPT Codex.

ChatGPT stelde voor eerst een versie te maken met minimale functionaliteit. Die bestond al uit een viertal verschillende bestanden die ik telkens, met de juiste naam, in dezelfde map moest bewaren. Bij foutmeldingen plakte ik die melding opnieuw in ChatGPT, dat een verbeterde versie van de bestanden maakte. Ja, dat kan behoorlijk omslachtig worden.

Met al dat knippen en plakken was ik zo al snel drie uur bezig. Het resultaat: een werkende app, maar wel een die er nogal belabberd uitzag en waar je, bijvoorbeeld, een activiteit wel kon toevoegen maar niet kon wissen of wijzigen.

Echt agentic werken

Wie verder wil gaan dan dat – zonder zelf te gaan programmeren – moet 'agentic' gaan werken: met een AI-systeem dat zelfstandig aan de slag gaat en je programmabestanden kan zien en zelf kan wijzigen. Dan gaat alles veel sneller en kun je eigenlijk haast alles bouwen. Je kunt de werking en de vormgeving eindeloos verfijnen. Maar je moet wel een beetje begrijpen waar je mee bezig bent.

Je hoort dezer dagen veel over Claude Code, maar ik opteerde voor het gelijkaardige ChatGPT Codex. Dat vereist de betaalversie van ChatGPT, ChatGPT Plus (23 euro per maand). Codex is een 'agent': je geeft nog altijd een instructie via een tekstprompt, maar dan gaat de AI zelfstandig verschillende stappen na elkaar zetten en zelf ook eigen wijzigingen voorstellen. Je moet alleen nog af en toe op 'ja' klikken.

Er zijn ook talloze gespecialiseerde vibecoding-instrumenten, zoals Cursor, Replit en Lovable. Die laatste twee heb ik vrij uitgebreid uitgeprobeerd, ze werken minstens even goed als Codex en hebben een rijkere interface. Het probleem is dat je er doorgaans niet in slaagt om een app helemaal af te werken met de gratis versie van Replit of Lovable, dus je moet betalen – en dan betaal ik liever voor ChatGPT Plus of Gemini Pro, zodat ik én kan vibecoden én een betere chatbot ter beschikking heb.

Mijn Gezinskalender besloot ik dus verder af te werken met ChatGPT Codex. Dat kan gewoon op het web, maar ik installeerde het programma ChatGPT Codex CLI. Zo had Codex rechtstreeks toegang tot de bestanden op mijn pc. En, simpel gezegd: wat toen gebeurde, lijkt wel toverij. Stap voor stap begon Codex mijn programma te verbeteren, op basis van mijn suggesties maar ook op eigen initiatief. Twintig minuten later leek de app eigenlijk klaar om in de app stores van Apple en Google aan te bieden (behalve dat het, strikt gesproken, geen echte app maar een Progressive Web App is: je kunt haar bewaren met een icoontje op je smartphone-scherm, maar je downloadt haar niet via de appstore).

In totaal heeft mijn afgewerkte Gezinskalender mij een halve dag werk gekost. Maar in die tijd heb ik enorm veel geleerd en bijvoorbeeld ook accounts op Supabase en Vercel gemaakt. Ik weet zeker dat ik een app van deze complexiteit een volgende keer in de helft van die tijd in elkaar gestoken krijg.

Voorkennis

Nog een tip voor wie zijn weg zoekt in vibecoding: in plaats van Kladblok te gebruiken, heb ik intussen een ander gratis programma van Microsoft geïnstalleerd, VS Code. Met Codex, VS Code en Github, soms nog gecombineerd met Supabase en Vercel, kun je haast alles bouwen. Letterlijk zonder een lijn code te schrijven, of zelfs maar te lezen. Tussen mijn eerste Javascript-game en de apps die ik nu met Codex bouw, moest ik wel heel wat kennis opbouwen over al die componenten. Ik schat dat ik daar twee of drie dagen over heb gedaan.

Hoe groot je voorkennis is, maakt een verschil. Ik heb in mijn jeugd wat leren programmeren, in de programmeertaal Basic (op een Sinclair ZX-81, heus!). Net voldoende om een beetje te begrijpen wat er aan het gebeuren is als die chatbot code genereert. Dat heeft mij zeker geholpen, maar mensen zonder enige programmeerkennis slagen erin om gelijkaardige apps te vibecoden.

Nog iets: agentic programmeren evolueert momenteel razendsnel. Wat zes maanden geleden uren werk kostte, komt nu in één keer uit ChatGPT gerold. Iets vrij complex als mijn Gezinskalender duurt nu enkele uren, maar kun je over een paar weken of maanden waarschijnlijk met twee-drie prompts genereren

T-REX HEADER

Gebruik de knoppen om te bewegen en te koppen!

High Score: 0

START SPEL

KOP!

LEES OOK

Echt waar: niks AI

LEES OOK

Mijn destructieve cyberhuwelijk met Claude

Inspiratie | De helpdesk | Helpdesk

Fout gezien? Meld het ons hier